



सो रिसेंटली हमने जावास्क्रिप्ट ...
TXT 1.54MB

make notes for me out of this transcript explain everything properly

React JS Complete Notes (Hindi → English Structured Notes)

Based on the transcript from Sheryians Coding School — "React JS Learn Everything" by Sarthak Sharma

PART 1: Introduction & Theory

What is React JS?

- A **JavaScript library** used to build UIs (User Interfaces)
- Created by **Meta (Facebook)** in **2013**
- One of the most in-demand skills in 2025 (even with AI boom)
- Focuses on **complex frontend** development

Why React Was Needed? (The Story)

Problem with vanilla JS + Facebook:

- Facebook's UI (notification count, likes, friend requests) needed to update dynamically
- Without React, the **entire page had to reload** just to update one small thing
- Poor user experience

React's Solution — Component-Based Architecture:

- Break the whole website into small independent **components**
- Only the component that changes gets re-rendered
- Example: YouTube — search bar stays same, only video area changes; Instagram — only notification count updates

Library vs Framework

Library	Framework
Provides a feature	Provides everything built-in
Example: GSAP (animation), Locomotive/Lenis (smooth scroll), React (UI)	Example: Angular, Next.js, Remix
Like buying a plot and building your own house	Like buying a ready-made flat — less customization

React is a **LIBRARY** (not a framework).

PART 2: JavaScript Prerequisites

Before React, you should know:

- Variables, Loops, Conditionals (if-else)
- Data types
- Functions, Arrow Functions, Return statements
- Arrays — `map` , `filter` , `reduce` , `forEach`
- Objects, Object literals
- **ES6 features** — `let` , `const` , arrow functions, template literals
- **Destructuring**
- **Asynchronous JS** — Promises, `fetch` API, `async/await`
- **Import/Export** (ES6 modules)

PART 3: Setting Up React

Tools Needed

- **VS Code** (code editor)
- **Live Server** extension

- **Dark Reader** (optional)
- **Node.js** (required for npm)

Creating a React Project

bash

```
npm create vite
```

- Choose: **React** → **JavaScript**
- Then:

bash

```
cd your-folder
```

```
npm install
```

```
npm run dev
```

Folder Structure

text

```
my-app/  
├─ node_modules/  
├─ public/  
├─ src/  
│   ├─ assets/  
│   ├─ App.jsx  
│   ├─ main.jsx  
│   ├─ index.css  
│   └─ App.css  
├─ index.html  
├─ package.json  
├─ vite.config.js  
└─ .gitignore
```

Hierarchy:

- `index.html` → contains `<div id="root">`
- `main.jsx` → creates root, renders `<App />`
- `App.jsx` → your main component

Important Extensions

- Simple React Snippets (by Burke Holland)
- ES7+ React/Redux/React-Native snippets
- Tailwind CSS IntelliSense

Useful Shortcuts

- `rfce` → React Functional Component Export
- `useState` snippet → `useState` (auto-import)

PART 4: Import & Export

Default Export

jsx

```
// app.jsx
const user = "Sarthak";
export default user;

// script.jsx
import anyName from './app';
console.log(anyName); // Sarthak
```

- Only **one** default export per file
- Import name can be **anything**

Named Export

jsx

```
// app.jsx
export const age = 89;
export const skills = ["Frontend", "Backend"];

// script.jsx
import { age, skills } from './app';
```

- Multiple named exports allowed
- Import name **must match** exactly

Module Type

html

```
<script type="module" src="./script.js"></script>
```

Required for import/export in vanilla JS.

PART 5: Real DOM vs Virtual DOM

Real DOM

- Actual HTML tree structure
- When something changes → **entire DOM re-renders** (slow)

Virtual DOM

- A **copy** of the real DOM (React maintains 1–3 copies)
- When changes happen:
 1. React updates the Virtual DOM first
 2. Compares with Real DOM (diffing)
 3. Only **changed parts** are updated in Real DOM

Benefits

- Faster performance
- Better user experience
- No unnecessary re-rendering
- Example: Instagram notification count updates without full page reload

PART 6: JSX (JavaScript XML)

JSX = HTML + JavaScript combination

Without JSX (Vanilla JS)

js

```
const h1 = document.createElement("h1");  
h1.innerHTML = "Hello from JS";  
document.body.appendChild(h1);
```

With JSX

jsx

```
const h1 = <h1>Hello from JSX</h1>;
```

Rules

- Only **ONE parent element** can be returned
- Use **fragments** `<>...</>` to wrap multiple elements
- Use `className` instead of `class` (class is a keyword in JS)
- Use `{ }` to embed JS expressions

Example

jsx

```
function App() {  
  const user = "Harsh";  
  const age = 89;  
  return (  
    <div>  
      <h1>Hello Guys, I am {user}</h1>  
      <h2>And I am {age} years old</h2>  
    </div>  
  );  
}  
  
export default App;
```

PART 7: Components

Component = a small, reusable piece of code for a specific feature.

Types

1. **Functional Components** (modern, preferred)
2. **Class Components** (older, obsolete)

Example

jsx

```
// Card.jsx
const Card = () => {
  return <div className="card">Hello, I am Card</div>;
};
export default Card;

// App.jsx
import Card from './components/Card';
function App() {
  return (
    <div>
      <Card />
      <Card />
      <Card />
    </div>
  );
}
```

Benefits

- Reusability
- Easy maintenance
- Only the changed component re-renders
- Scalability

PART 8: Props (Properties)

Props = data passed from **parent** → **child**. Data flow is **one-way (top to bottom)**.

Passing Props

jsx

```
<Card user="Aman" age={18} />
<Card user="Sarthak" age={28} />
```

Receiving Props

jsx

```
const Card = (props) => {
  console.log(props); // { user: "Aman", age: 18 }
  return (
    <div>
      <h1>{props.user}</h1>
      <p>{props.age}</p>
    </div>
  );
};
```

Rules

- Strings: `user="Aman"`
- Numbers/arrays/objects/variables: `{ }` — `age={18}`
- **Cannot** change props inside child
- **Data only flows downward**

Props Drilling

- Passing props through **multiple levels** of components
- Problem: messy for large apps
- Solution: **Context API** or **Redux**



PART 9: CSS in React

1. Global CSS (index.css)

- Applies to entire app
- OK for small projects

2. Component-based CSS

- Separate CSS file per component
- Import in component
- **Problem:** class name collisions

3. CSS Modules

jsx

```
// Button.module.css
.btn { background: red; }

// Button.jsx
import styles from './Button.module.css';
<button className={styles.btn}>Click</button>
```

- Scoped styles (no collisions)
- Class names get hashed

4. Tailwind CSS (Utility-first)

bash

```
npm install tailwindcss @tailwindcss/vite
```

Configure vite.config.js :

js

```
import tailwindcss from '@tailwindcss/vite';
export default defineConfig({
```

```
plugins: [react(), tailwindcss()],  
});
```

Import in `index.css` :

css

```
@import "tailwindcss";
```

Usage:

jsx

```
<div className="bg-red-600 h-screen text-white">Hello</div>
```

PART 10: Folder Structures

Feature-Based

- One component per **feature**
- e.g., `Navbar/` , `Card/` , `Footer/`

Atomic Design

- Break things into **smallest reusable units**
- Button, Heading, etc. as separate components

Best Practice: Combination of both

text

components/

├─ Navbar/

| └─ Navbar.jsx

| └─ Navbar.module.css

├─ Card/

| └─ Card.jsx

| └─ Card.module.css

PART 11: Functions & Events

Calling Functions

jsx

```
const btnClicked = () => {  
  console.log("Button clicked");  
};
```

```
<button onClick={btnClicked}>Click</button> //  Correct
```

```
<button onClick={btnClicked()}>Click</button> //  Runs immediately!
```

Common Events

- `onClick` , `onDoubleClick`
- `onMouseEnter` , `onMouseLeave` , `onMouseMove`
- `onChange` (input)
- `onSubmit` (form)
- `onScroll` , `onWheel`

Inline Function

jsx

```
<button onClick={() => console.log("Clicked")}>Click</button>
```

Getting Event Object

jsx

```
const inputChanging = (e) => {  
  console.log(e.target.value); // current input value  
};
```

```
<input onChange={inputChanging} />
```



PART 12: Hooks (Overview)

Hooks are **special functions** in React that let you "hook into" React features.

Hook	Purpose
useState	Manage state (variables that trigger re-render)
useEffect	Side effects (API calls, DOM manipulation, timers)
useContext	Global state management (avoid props drilling)
useReducer	Complex state management (Redux-like)
useRef	Reference to DOM elements / mutable values
useMemo	Memoization (performance)
useCallback	Memoize functions (performance)



PART 13: useState (Basic)

Why Not Regular Variables?

jsx

```
let a = 20;  
const changeA = () => { a = 30; }; // ❌ UI won't update
```

React needs to know when to re-render.

Using useState

jsx

```
import { useState } from 'react';  
  
const [name, setName] = useState(0);  
  
const increment = () => setName(name + 1);  
const decrement = () => setName(name - 1);
```

```
<button onClick={increment}>Increase</button>
<button onClick={decrement}>Decrease</button>
<h1>{name}</h1>
```

Key Concepts

- name → **read-only** variable
- setName → **function** to change value
- Changing state → **triggers re-render**
- Setting **same value** → **no re-render**

Async Nature of setState

jsx

```
setName(name + 1);
console.log(name); // ❌ Still shows old value
```

- State updates are **asynchronous**
- Use **previous value** for updates:

jsx

```
setName((prev) => prev + 1);
```

Updating Objects

jsx

```
const [user, setUser] = useState({ name: "Sarthak", age: 20 });

const update = () => {
  const newUser = { ...user, name: "Aman" };
  setUser(newUser);
};
```

Updating Arrays

jsx

```
const [arr, setArr] = useState([10, 20, 30]);

const addItem = () => {
  const newArr = [...arr];
  newArr.push(99);
  setArr(newArr);
};
```



PART 14: Forms & Two-Way Binding

Form Handling (Preventing Reload)

jsx

```
const handleSubmit = (e) => {
  e.preventDefault(); // Stops default reload behavior
  console.log("Form submitted");
};

<form onSubmit={handleSubmit}>
  <input type="text" />
  <button type="submit">Submit</button>
</form>
```

Two-Way Binding

jsx

```
const [title, setTitle] = useState("");

<input
  value={title}
  onChange={(e) => setTitle(e.target.value)}
/>
```

- value → displays state
- onChange → updates state
- Both directions → **Two-way binding**

On Submit

jsx

```
const handleSubmit = (e) => {  
  e.preventDefault();  
  console.log(title);  
  setTitle(""); // clear input  
};
```

PART 15: Local Storage

What is Local Storage?

- Browser's memory for storing small data
- Persists even after browser close
- **Session Storage** → cleared when browser closes

Methods

js

```
localStorage.setItem("user", "Sarthak"); // Set  
localStorage.getItem("user");           // Get  
localStorage.removeItem("user");         // Remove one  
localStorage.clear();                    // Clear all
```

Storing Objects/Arrays

js

```
// Save  
localStorage.setItem("user", JSON.stringify(user));  
  
// Retrieve  
const user = JSON.parse(localStorage.getItem("user"));
```



PART 16: API Handling

Architecture

text

Database → Backend → API → Frontend

Two Methods

1. **fetch** (built-in)
2. **axios** (third-party, preferred)

Install Axios

bash

```
npm install axios
```

Using fetch

jsx

```
const getData = async () => {  
  const response = await fetch('https://jsonplaceholder.typicode.com/todos/1');  
  const data = await response.json();  
  console.log(data);  
};
```

Using axios

jsx

```
import axios from 'axios';  
  
const getData = async () => {  
  const { data } = await axios.get('https://jsonplaceholder.typicode.com/users');  
  console.log(data);  
};
```

Common HTTP Methods

Method	Purpose
GET	Fetch data
POST	Send data
PATCH	Update data
DELETE	Delete data

Displaying API Data

jsx

```
const [users, setUsers] = useState([]);

const getData = async () => {
  const { data } = await axios.get(URL);
  setUsers(data);
};

useEffect(() => { getData(); }, []);

return (
  <div>
    {users.map((user, idx) => (
      <h3 key={idx}>{user.name}</h3>
    ))}
  </div>
);
```

PART 17: useEffect

Why useEffect?

- React's rendering process = UI creation
- Some tasks run **parallel/side-by-side** — API calls, timers, DOM manipulation
- These are called **Side Effects** — handled by `useEffect`

Mental Model

- Main road = React rendering process
- Footpath = useEffect (side tasks run alongside)

Syntax

jsx

```
import { useEffect } from 'react';
```

```
useEffect(() => {  
  console.log("Running");  
}, []); // dependency array
```

Dependency Array Behavior

Array	Runs
Not passed	After every render
[]	Only once (on mount)
[a]	When a changes
[a, b]	When a or b changes

Example

jsx

```
const [a, setA] = useState(0);  
const [b, setB] = useState(100);  
  
useEffect(() => {  
  console.log("A changed");  
}, [a]); // only runs when a changes
```



PART 18: Notes App Project (Practice)

Features built:

- Form with title + details inputs
- Two-way binding
- Adding notes
- Displaying notes via `map`
- Deleting notes via `splice`
- Responsive UI with Tailwind
- Local state management

Delete Logic:

jsx

```
const deleteNote = (idx) => {  
  const copyTask = [...task];  
  copyTask.splice(idx, 1);  
  setTask(copyTask);  
};
```



PART 19: Gallery Project (API Practice)

Features:

- Fetch images from Lorem Picsum API
- Pagination (prev/next buttons)
- Dynamic routes
- Loading state

Pagination Logic:

jsx

```
const [index, setIndex] = useState(1);  
  
// Next  
if (index < 10) setIndex(index + 1);
```

```
// Previous
if (index > 1) setIndex(index - 1);
```

API URL with dynamic index:

js

```
`https://picsum.photos/v2/list?page=${index}&limit=10`
```



PART 20: React Router DOM

Install

bash

```
npm install react-router-dom
```

Types of Routers

Router	Use Case
BrowserRouter	Most common, client-side routing
HashRouter	URL has # , doesn't hit server
MemoryRouter	React Native, static apps
StaticRouter	Static site generation (Next.js)

Setup

jsx

```
// main.jsx
import { BrowserRouter } from 'react-router-dom';
<BrowserRouter>
  <App />
</BrowserRouter>
```

Basic Routes

jsx

```
import { Routes, Route } from 'react-router-dom';

<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/about" element={<About />} />
  <Route path="/contact" element={<Contact />} />
  <Route path="*" element={<NotFound />} />
</Routes>
```

Link (no reload)

jsx

```
import { Link } from 'react-router-dom';

<Link to="/">Home</Link>
<Link to="/about">About</Link>
```

Nested Routes

jsx

```
<Route path="/product" element={<Product />}>
  <Route path="men" element={<Mens />} />
  <Route path="women" element={<Womens />} />
</Route>

// In Product.jsx
import { Outlet } from 'react-router-dom';
return (
  <div>
    <h1>Product Page</h1>
    <Outlet /> {/* Child routes render here */}
  </div>
);
```

Dynamic Routes

jsx

```
<Route path="/courses/:courseId" element={}<CourseDetail /> />
```

```
// In CourseDetail.jsx
import { useParams } from 'react-router-dom';
const { courseId } = useParams();
console.log(courseId); // whatever is in URL
```

useNavigate

jsx

```
import { useNavigate } from 'react-router-dom';

const navigate = useNavigate();

<button onClick={() => navigate('/')}>Go Home</button>
<button onClick={() => navigate(-1)}>Back</button>
<button onClick={() => navigate(1)}>Next</button>
```

PART 21: Context API

Why Context API?

- **Problem:** Props Drilling (passing data through many levels)
- **Solution:** Centralize data — any component can access it directly

Real-Life Analogy

- Instead of handing water to each person one-by-one (props drilling)
- Put water in a **swimming pool** and let everyone access it directly (Context)

3 Steps

1. **Create** context
2. **Provide** data
3. **Consume** data (useContext)

Example

1. Create Context (ThemeContext.jsx)

jsx

```
import { createContext } from 'react';  
export const ThemeDataContext = createContext();
```

2. Provide Data (App.jsx)

jsx

```
import { ThemeDataContext } from '../context/ThemeContext';  
import { useState } from 'react';  
  
function App() {  
  const [theme, setTheme] = useState("light");  
  
  return (  
    <ThemeDataContext.Provider value={{ theme, setTheme }}>  
      <Navbar />  
    </ThemeDataContext.Provider>  
  );  
}
```

3. Consume Data (Navbar.jsx)

jsx

```
import { useContext } from 'react';  
import { ThemeDataContext } from '../context/ThemeContext';  
  
const { theme, setTheme } = useContext(ThemeDataContext);  
  
<button onClick={() => setTheme("dark")}>Change Theme</button>  
<h1>Theme: {theme}</h1>
```

Benefits

- No props drilling
- Global state
- Easy to update from anywhere

- Perfect for themes, user data, auth status

BONUS: Children Concept

When you write:

```
jsx

<Navbar>
  <h1>Hello</h1>
</Navbar>
```

The wrapped content is accessible via `props.children` :

```
jsx

const Navbar = (props) => {
  console.log(props.children); // <h1>Hello</h1>
  return <div>{props.children}</div>;
};
```

- If no children → undefined
- If one child → object
- If multiple → array

FINAL SUMMARY

What You Learned

- ✓ What React is & why it's used
- ✓ Component-based architecture
- ✓ JSX, Props, State
- ✓ useState (basic + advanced)
- ✓ useEffect with dependencies
- ✓ Forms + Two-way binding
- ✓ Local Storage
- ✓ API handling (fetch/axios)

- ✓ React Router DOM (basic, nested, dynamic)
- ✓ Context API
- ✓ Multiple practice projects (Notes, Gallery, Website)

Next Steps

- Practice all projects
- Learn **Redux** (advanced state management)
- Learn **TypeScript**
- Build **real-life large projects**
- Learn **Next.js** for full-stack
- Interview prep (theoretical + tasks)

Quick Reference Cheatsheet

jsx

// Component

```
const MyComp = () => <div>Hello</div>;  
export default MyComp;
```

// State

```
const [val, setVal] = useState(0);
```

// Effect

```
useEffect(() => { /* Logic */ }, [dep]);
```

// Props

```
<Child name="Aman" age={20} />  
const Child = ({ name, age }) => <p>{name} {age}</p>;
```

// Context

```
const Ctx = createContext();  
<Ctx.Provider value={data}>...</Ctx.Provider>  
const data = useContext(Ctx);
```

// Router

```
<BrowserRouter>  
  <Routes>  
    <Route path="/" element={<Home />} />  
  </Routes>  
</BrowserRouter>
```

```
<Route path="/user/:id" element={<User />} />
</Routes>
</BrowserRouter>
const { id } = useParams();
const nav = useNavigate();

// API
const { data } = await axios.get(URL);
```

Instructor: Sarthak Sharma

Channel: Sheryians Coding School

Total Content: ~12+ hours comprehensive React tutorial with projects

🎯 **Key Takeaway:** React's power lies in **components**, **state management**, and **efficient rendering** via Virtual DOM. Practice every project to master it!

give me interview questions with answer

🎯 React JS Interview Questions & Answers

Based on the complete transcript — organized by difficulty level

● SECTION 1: Basics & Theory

Q1. What is React JS?

Answer:

React JS is an **open-source JavaScript library** used for building user interfaces (UIs), especially for single-page applications. It was created by **Meta (formerly Facebook)** in **2013**. React allows developers to create reusable UI components and efficiently update the DOM using a Virtual DOM.

Key points:

- It's a **library**, not a framework
- Component-based architecture
- Uses Virtual DOM for performance

- One-way data binding

Q2. What is the difference between a Library and a Framework?

Answer:

Library	Framework
Provides a specific feature	Provides complete architecture
You control the flow	Framework controls the flow
Example: React, GSAP, Axios	Example: Angular, Next.js, Remix
Like buying a plot & building your house	Like buying a ready-made flat

React is a library because it only handles the view layer — you decide how to structure the rest.

Q3. What is the difference between Real DOM and Virtual DOM?

Answer:

Real DOM	Virtual DOM
Actual HTML tree in browser	JavaScript copy of Real DOM
Updating is slow	Updating is fast
Entire DOM re-renders on change	Only changed parts re-render
Direct manipulation	React handles manipulation

How Virtual DOM works:

1. React creates a Virtual DOM copy
2. On state change, a new Virtual DOM is created
3. React compares (diffs) old vs new Virtual DOM

4. Only the **changed elements** are updated in Real DOM

Benefit: Better performance, smoother UX.

Q4. What is JSX?

Answer:

JSX stands for **JavaScript XML**. It's a syntax extension that allows you to write **HTML-like code inside JavaScript**.

jsx

```
const element = <h1>Hello, World!</h1>;
```

Rules:

- Must return **one parent element**
- Use `className` instead of `class`
- Use `{ }` to embed JS expressions
- Use fragments `<>...</>` to wrap multiple elements

Without JSX (vanilla JS):

js

```
const h1 = document.createElement("h1");  
h1.innerHTML = "Hello";  
document.body.appendChild(h1);
```

Q5. What is a Component in React?

Answer:

A **component** is a small, reusable, independent piece of code that represents a part of the UI. There are two types:

1. **Functional Components** (modern, preferred)
2. **Class Components** (older, obsolete)

Example:

```
jsx

const Card = () => {
  return <div className="card">Hello, I am Card</div>;
};
export default Card;
```

Benefits: Reusability, easy maintenance, better performance, scalability.

● SECTION 2: Props & State

Q6. What are Props in React?

Answer:

Props (Properties) are read-only data passed from a **parent component** to a **child component**. Data flow is **one-way (top to bottom)**.

```
jsx

// Parent
<Card user="Aman" age={18} />

// Child
const Card = (props) => {
  return (
    <div>
      <h1>{props.user}</h1>
      <p>{props.age}</p>
    </div>
  );
};
```

Key rules:

- Props are **immutable** (cannot be changed inside child)
- Strings pass as `user="Aman"`
- Numbers/objects/arrays pass as `{ }` — `age={18}`
- Data **cannot flow upward**

Q7. What is Props Drilling?

Answer:

Props Drilling is the process of passing props through **multiple levels** of components, even when intermediate components don't need them.

Example:

text

App → Navbar → Sidebar → UserProfile → Username

Each intermediate component has to forward the prop, making the code messy.

Solution: Context API or Redux.

Q8. What is State in React?

Answer:

State is a built-in object that holds data that may change over the lifetime of a component. When state changes, React **re-renders** the component.

jsx

```
const [count, setCount] = useState(0);
```

- count → read-only current value
- setCount → function to update state

Difference between Props & State:

Props	State
Passed from parent	Defined inside component
Immutable	Mutable
Read-only	Can be updated

Props	State
External	Internal

Q9. What is useState Hook?

Answer:

`useState` is a React Hook that lets you add state to functional components.

jsx

```
import { useState } from 'react';

const [name, setName] = useState("Sarthak");

setName("Aman"); // updates state, triggers re-render
```

Rules:

- Always called at the **top level** of component
- Never inside loops, conditions, or nested functions
- Returns an **array**: `[currentValue, setterFunction]`

Q10. Why does setState not update immediately?

Answer:

`setState` is **asynchronous**. When you call it:

jsx

```
setName("Aman");
console.log(name); // still shows old value
```

React batches multiple state updates for performance. The state updates **after** the current function completes.

Fix — use previous value:

jsx

```
setCount((prev) => prev + 1);
```

Q11. What happens if you set the same state value again?

Answer:

If the new state is **equal to the current state** (using `Object.is`), React **skips the re-render** for optimization.

jsx

```
setName("Aman"); // first time → re-render  
setName("Aman"); // second time → NO re-render
```

SECTION 3: useEffect

Q12. What is useEffect and why is it used?

Answer:

`useEffect` is a Hook used to perform **side effects** in functional components. Side effects include:

- API calls
- DOM manipulation
- Setting timers/intervals
- Event listeners
- Subscriptions

Analogy: React's rendering is the main road (truck). `useEffect` is the footpath — side tasks run parallel to the main process.

Syntax:

jsx

```
useEffect(() => {  
  // side effect code  
}, [dependencies]);
```

Q13. What is a Dependency Array in useEffect?

Answer:

Dependency Array	Behavior
Not passed	Runs after every render
[] (empty)	Runs only once (on mount)
[a]	Runs when a changes
[a, b]	Runs when a or b changes

Example:

jsx

```
useEffect(() => {  
  console.log("A changed");  
}, [a]);
```

Q14. What is the difference between useEffect and a normal function?

Answer:

Normal Function	useEffect
Runs during render	Runs after render
Blocks rendering	Runs side-by-side
Runs every time	Runs based on dependencies

Normal Function

useEffect

Not for side effects

Specifically for side effects

● SECTION 4: Forms & Events

Q15. How do you handle forms in React?

Answer:

Using **controlled components** with two-way binding:

jsx

```
const [title, setTitle] = useState("");

const handleSubmit = (e) => {
  e.preventDefault(); // prevents page reload
  console.log(title);
  setTitle("");
};

<form onSubmit={handleSubmit}>
  <input
    value={title}
    onChange={(e) => setTitle(e.target.value)}
  />
  <button type="submit">Submit</button>
</form>
```

Q16. What is Two-Way Binding?

Answer:

Two-way binding means the **input value** and **state** are **synchronized** in both directions:

- **State → Input:** `value={title}`
- **Input → State:** `onChange={(e) => setTitle(e.target.value)}`

Any change in the input updates the state, and any change in state updates the input.

Q17. Why do we use `e.preventDefault()` in forms?

Answer:

Forms have a **default behavior** — when submitted, they reload the page. Since React is used for single-page applications, reloading breaks the UX.

`e.preventDefault()` **stops** the default reload behavior, letting you handle the submission manually.

jsx

```
const handleSubmit = (e) => {  
  e.preventDefault();  
  // custom logic  
};
```

Q18. What is the difference between `onClick` and `onChange`?

Answer:

onClick	onChange
Fires on click	Fires on value change
Used for buttons, divs	Used for inputs, selects, textareas
<code>e</code> contains click details	<code>e.target.value</code> gives current value

SECTION 5: Hooks Advanced

Q19. What are Hooks? What are the rules of Hooks?

Answer:

Hooks are special functions that let you "hook into" React features from functional components.

Rules:

1. Only call Hooks at the **top level** — not inside loops, conditions, or nested functions
2. Only call Hooks from **React function components** or **custom Hooks**
3. Hooks must be called in the **same order** every render

Common Hooks:

- `useState` , `useEffect` , `useContext` , `useReducer` , `useRef` , `useMemo` , `useCallback`

Q20. What is useContext?

Answer:

`useContext` is a Hook that lets you **consume** context values without prop drilling.

3 Steps:

1. **Create** context: `createContext()`
2. **Provide** value: `<Context.Provider value={data}>`
3. **Consume**: `const data = useContext(Context)`

Example:

```
jsx

// Create
export const ThemeContext = createContext();

// Provide
<ThemeContext.Provider value={{ theme, setTheme }}>
  <App />
</ThemeContext.Provider>

// Consume
const { theme, setTheme } = useContext(ThemeContext);
```

Q21. What is useRef?

Answer:

`useRef` returns a **mutable object** with a `.current` property that persists across renders.

Two main uses:

1. Access DOM elements directly:

```

jsx

const inputRef = useRef(null);


```

2. Store mutable values that don't trigger re-renders.

Q22. What is useMemo and useCallback?

Answer:
Both are performance optimization Hooks.

useMemo	useCallback
Memoizes a value	Memoizes a function
Returns cached value	Returns cached function
Prevents expensive recalculations	Prevents unnecessary re-creations

```

jsx

const memoizedValue = useMemo(() => computeExpensive(a, b), [a, b]);
const memoizedFn = useCallback(() => doSomething(a), [a]);
    
```

Q23. What is useReducer?

Answer:
useReducer is an alternative to useState for **complex state logic**. It's like a mini version of Redux.

```

jsx
    
```

```
const reducer = (state, action) => {
  switch (action.type) {
    case 'increment': return { count: state.count + 1 };
    case 'decrement': return { count: state.count - 1 };
    default: return state;
  }
};

const [state, dispatch] = useReducer(reducer, { count: 0 });
dispatch({ type: 'increment' });
```

When to use:

- Complex state with multiple sub-values
- Next state depends on previous state
- Many state transitions

● SECTION 6: React Router

Q24. What is React Router and why is it used?

Answer:

React Router is a **routing library** for React applications. React doesn't have built-in routing, so we use React Router to navigate between pages **without reloading** the browser.

Types of Routers:

- **BrowserRouter** — most common, uses History API
- **HashRouter** — uses # in URL
- **MemoryRouter** — stores in memory (React Native)
- **StaticRouter** — for static site generation

Q25. How do you set up React Router?

Answer:

```
bash
```

```
npm install react-router-dom
```

```
jsx
```

```
// main.jsx
import { BrowserRouter } from 'react-router-dom';
<BrowserRouter>
  <App />
</BrowserRouter>
```

```
// App.jsx
import { Routes, Route } from 'react-router-dom';

<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/about" element={<About />} />
  <Route path="*" element={<NotFound />} />
</Routes>
```

Q26. What is the difference between Link and anchor <a> tag?

Answer:

<a> tag	<Link>
Causes full page reload	No reload (client-side)
Hits server	Handled by React Router
Slower UX	Fast SPA navigation

```
jsx
```

```
<Link to="/about">About</Link>
```

Q27. What are Nested Routes?

Answer:

Nested routes allow you to render **child routes inside a parent route**.

jsx

```
<Route path="/product" element={<Product />}>
  <Route path="men" element={<Mens />} />
  <Route path="women" element={<Womens />} />
</Route>

// Product.jsx
import { Outlet } from 'react-router-dom';
return (
  <div>
    <h1>Product Page</h1>
    <Outlet /> { /* Child routes render here */ }
  </div>
);
```

URLs: /product/men , /product/women

Q28. What are Dynamic Routes?**Answer:**

Dynamic routes use **URL parameters** to render content based on the URL.

jsx

```
<Route path="/courses/:courseId" element={<CourseDetail />} />

// CourseDetail.jsx
import { useParams } from 'react-router-dom';
const { courseId } = useParams();
console.log(courseId); // "frontend", "backend", etc.
```

Example URLs: /courses/frontend , /courses/3js

Q29. What is useNavigate?

Answer:

`useNavigate` is a Hook used for **programmatic navigation** — navigating in code, not by clicking a link.

jsx

```
import { useNavigate } from 'react-router-dom';
const navigate = useNavigate();

navigate('/');      // go to home
navigate(-1);       // go back
navigate(1);        // go forward
```

Use case: After login, redirect user to dashboard.

● SECTION 7: API & Data Handling

Q30. How do you handle API calls in React?

Answer:

Two common methods:

1. `fetch` (built-in):

jsx

```
const getData = async () => {
  const res = await fetch(URL);
  const data = await res.json();
  console.log(data);
};
```

2. `axios` (third-party, preferred):

jsx

```
import axios from 'axios';

const getData = async () => {
  const { data } = await axios.get(URL);
```

```
console.log(data);  
};
```

Why axios is preferred:

- Better error handling
- Automatic JSON parsing
- Interceptors support
- Request/response transformation

Q31. What is the difference between GET, POST, PATCH, and DELETE?

Answer:

Method	Purpose
GET	Fetch data from server
POST	Send new data to server
PATCH	Update existing data (partial)
PUT	Replace existing data (full)
DELETE	Remove data

jsx

```
axios.get(URL);           // fetch  
axios.post(URL, data);    // create  
axios.patch(URL, data);   // update  
axios.delete(URL);        // delete
```

Q32. What is the difference between async/await and .then()?

Answer:

async/await**.then()**

Cleaner, readable

Chained, can get messy

Uses try/catch for errors

Uses .catch()

Synchronous-looking code

Callback-based

Newer syntax

Older syntax

jsx

// async/await

```
const getData = async () => {
  try {
    const { data } = await axios.get(URL);
    console.log(data);
  } catch (err) {
    console.log(err);
  }
};
```

// .then()

```
axios.get(URL)
  .then((res) => console.log(res.data))
  .catch((err) => console.log(err));
```

SECTION 8: Styling

Q33. What are the different ways to style React components?

Answer:

1. **Global CSS** (`index.css`) — applies to entire app
2. **Component CSS** — separate file per component
3. **CSS Modules** (`Button.module.css`) — scoped class names
4. **Inline styles** — `style={{ color: 'red' }}`
5. **Tailwind CSS** — utility-first framework
6. **Styled-components** — CSS-in-JS

CSS Modules example:

jsx

```
import styles from './Button.module.css';  
<button className={styles.btn}>Click</button>
```

Q34. What is Tailwind CSS and why is it popular?

Answer:

Tailwind CSS is a **utility-first CSS framework** — instead of writing custom CSS, you use predefined utility classes.

jsx

```
<div className="bg-red-600 h-screen text-white p-4">Hello</div>
```

Benefits:

- No context switching between HTML & CSS
- No unused CSS in production
- Highly customizable
- Mobile-first design
- Fast development

Setup:

bash

```
npm install tailwindcss @tailwindcss/vite
```

Q35. What is the difference between className and class in React?

Answer:

- In HTML: `<div class="card">`
- In React/JSX: `<div className="card">`

Why? Because `class` is a **reserved keyword** in JavaScript. JSX compiles to JavaScript, so `class` would conflict.

● SECTION 9: Advanced Concepts

Q36. What is the Virtual DOM and how does it improve performance?

Answer:

The Virtual DOM is a **JavaScript representation** of the Real DOM.

How it improves performance:

1. When state changes, React creates a **new Virtual DOM**
2. React **diffs** old vs new Virtual DOM
3. Only the **changed elements** are updated in Real DOM
4. Browser avoids expensive full re-renders

Real-life analogy: Instead of repainting an entire wall for one scratch, you touch up only that spot.

Q37. What is the difference between Controlled and Uncontrolled Components?

Answer:

Controlled	Uncontrolled
Value controlled by React state	Value managed by DOM
Uses <code>value + onChange</code>	Uses <code>ref</code>
Two-way binding	One-way
More code, more control	Less code, less control

jsx

```
// Controlled
<input value={name} onChange={(e) => setName(e.target.value)} />

// Uncontrolled
<input ref={inputRef} />
```

Q38. What is the difference between useState and useReducer?

Answer:

useState	useReducer
Simple state	Complex state
Direct update	Dispatch actions
Easier to understand	More structured
For 1-2 values	For multiple related values

When to use useReducer:

- State has multiple sub-values
- Next state depends on previous
- Many state transitions
- Logic is complex

Q39. What is Context API and when should you use it?

Answer:

Context API is a way to share data **globally** across components without prop drilling.

When to use:

- Theme (dark/light mode)
- User authentication
- Language/localization

- Shopping cart
- Any global state needed by many components

3 Steps:

1. `createContext()`
2. `<Context.Provider value={...}>`
3. `useContext(Context)`

Q40. What is Local Storage vs Session Storage?

Answer:

Local Storage	Session Storage
Persists forever	Cleared on browser close
~5-10 MB	~5 MB
Same origin access	Same origin access
For long-term data	For temporary data

Methods:

js

```
localStorage.setItem("user", "Sarthak");  
localStorage.getItem("user");  
localStorage.removeItem("user");  
localStorage.clear();
```

Storing objects:

js

```
localStorage.setItem("user", JSON.stringify(user));  
const user = JSON.parse(localStorage.getItem("user"));
```



SECTION 10: Scenario-Based Questions

Q41. How do you optimize a React application?

Answer:

1. **useMemo** — memoize expensive calculations
2. **useCallback** — memoize functions
3. **React.memo** — prevent unnecessary re-renders
4. **Code splitting** — lazy load components
5. **Lazy loading** — `React.lazy()` + `Suspense`
6. **Virtualization** — for long lists
7. **Avoid inline functions** in props
8. **Key prop** — use unique keys in lists

Q42. Why is the "key" prop important in lists?

Answer:

React uses the `key` prop to **identify which items have changed**, been added, or removed.

jsx

```
{users.map((user, idx) => (  
  <Card key={idx} user={user} />  
))}
```

Without key: React re-renders everything → poor performance

With key: React updates only changed items → efficient

Best practice: Use a **unique ID**, not array index.

Q43. What happens if you call `useState` inside a condition?

Answer:

It **breaks the Rules of Hooks** and causes an error.

Why? React relies on the **order** of Hook calls to match state with the correct Hook. If a Hook is conditionally called, the order changes between renders → React loses track.

jsx

```
// ❌ Wrong
if (condition) {
  const [x, setX] = useState(0);
}
```

```
// ✅ Correct
const [x, setX] = useState(0);
if (condition) { /* use x */ }
```

Q44. How would you build a "dark mode" feature in React?

Answer:

Use Context API + localStorage:

jsx

```
// Context
const ThemeContext = createContext();

// Provider
const [theme, setTheme] = useState(
  localStorage.getItem("theme") || "light"
);

useEffect(() => {
  localStorage.setItem("theme", theme);
}, [theme]);

<ThemeContext.Provider value={{ theme, setTheme }}>
  <App />
</ThemeContext.Provider>

// Consumer
const { theme, setTheme } = useContext(ThemeContext);
<button onClick={() => setTheme(theme === "light" ? "dark" : "light")}>
  Toggle Theme
</button>
```

Q45. How do you handle errors in API calls?

Answer:

Use **try/catch** with **async/await**:

jsx

```
const getData = async () => {
  try {
    const { data } = await axios.get(URL);
    setUsers(data);
  } catch (err) {
    console.error("Error:", err.message);
    setError("Failed to load data");
  } finally {
    setLoading(false);
  }
};
```

Best practices:

- Show error UI to user
- Log errors for debugging
- Use `finally` to reset loading state
- Consider error boundaries for component errors

BONUS: Rapid-Fire Questions

Question	Answer
React is a library or framework?	Library
Who created React?	Meta (Facebook), 2013
What is JSX?	JavaScript XML
Default export per file?	1
Named exports per file?	Multiple

Question	Answer
Data flow direction in React?	Top to bottom (one-way)
Immutable in React?	Props
Mutable in React?	State
Hook for side effects?	useEffect
Hook for global state?	useContext
Hook for DOM reference?	useRef
Hook for performance?	useMemo, useCallback
Router package name?	react-router-dom
Default router type?	BrowserRouter
URL parameter hook?	useParams
Navigation hook?	useNavigate
Form reload prevention?	e.preventDefault()
Storing objects in localStorage?	JSON.stringify / JSON.parse
Element uniqueness in lists?	key prop
Utility-first CSS?	Tailwind CSS



FINAL TIPS FOR INTERVIEW

1. **Understand concepts, don't memorize** — interviewers ask "why"
2. **Practice coding** — build small components live
3. **Know the difference** between similar concepts (props/state, useMemo/useCallback)
4. **Explain with examples** — real-life analogies help
5. **Ask clarifying questions** before coding
6. **Mention performance** — Virtual DOM, memoization, lazy loading

7. **Know the trade-offs** — when to use Context vs Redux vs useState

8. **Be honest** — if you don't know, say "I'd look it up"

All the best for your interview! 🚀